

COMPILER DESIGN

(22CS2235F)

STUDENT ID:2420030332 ACADEMIC YEAR: 2025-26

STUDENT NAME: Kolla Bhavana

Table of Contents

1. Session 01: Removing white spaces and comments Using LEX Tool 1
2. Session 02: Lexer Tokenization Process focusing on keywords using LEX tool 10
3. Session 03: Automating Lexical Analyzer Generation Using LEX Tool 21
4. Session 04: Elimination of Left Recursion from given CFG 30
5. Session 05: Implement Program to compute the FIRST set from the CFG 40
6. Session 06: Implement Program to compute the FOLLOW set from the CFG.....49
7. Session 07: Implement Program for Constructing LL(1) Parsing table.....59
8. Session 08: Implement Program for Shift-Reduce parser.....64
9. Session 09: Implementation of LR parser.....69
10. Session 10: Implement a System to generate and interpret quadruples 73
11. Session 11: Implement code optimization technique.....78
12. Session 12: Implement a module for generating intermediate code 83
13. Session 13: Implement basic error detection and reporting system.....88

A.Y. 2024-2025 LAB/SKILL CONTINUOUS EVALUATION

S.No	Date	Experiment Name	Pre-Lab (10M)	In-Lab (25M)			Post-Lab (10M)	Viva Voce (5M)	Total (50M)	Faculty Signature
				Program/ Procedure (5M)	Data and Results (10M)	Analysis and Inference (10M)				
1.		Removing white spaces and comments Using LEX Tool								
2.		Lexer Tokenization Process focusing on keywords using LEX tool								
3.		Automating Lexical Analyzer Generation Using LEX Tool								
4.		Elimination of Left Recursion from given CFG								
5.		Program to compute FIRST set for given CFG								
6.		Program to compute FOLLOW set for given CFG								
7.		Construction Of LL(1) Parsing table								
8.		Build Shift-Reduce parser								
9.		Implement of LR parser								
10.		Implement a System to generate and interpret quadruples								
11.		Implement code optimization technique								
12.		Implement a module for generating intermediate code								
13.		Implement basic error detection and reporting system								

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 1:

Experiment Title: Removing white spaces and comments Using LEX Tool.

Aim/Objective:

The aim of this lab session is to introduce students to programming for some small program in a HLL, such as C, that enables them to understand the working of small subset of the Lexical analyser. This familiarity will enable them to build confidence, and understanding of the overall module of the lexical analysis (scanning).

Description:

In this lab session, students will learn about

- To understand how to eliminate white space and comments

Pre-Requisites:

Students should have a basic understanding of the C language, and the ability to program using different constructs of the same.

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

#source Code

```
%%writefile whitespace.c
#include <stdio.h>

int main()
{
   char str[100];
   int i;

   printf("Enter any statement: ");
   fgets(str, sizeof(str), stdin);

   printf("Statement without spaces: ");

   for(i = 0; str[i] != '\0'; i++)
   {
      if(str[i] != ' ')
      {
         printf("%c", str[i]);
      }
   }
___
   return 0;
}
```

Runing step:

```
!gcc whitespace.c -o whitespace
!./whitespace
```

Output:

```
Enter any statement: Welcome to Compiler Design Lab
Statement without spaces: WelcometoCompilerDesignLab
```

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 2:
Experiment Title: Lexer Tokenization Process focusing on keywords using LEX tool.

Aim/Objective:

The aim of this lab session is to introduce students to programming a scanner for some small program (set of input strings) in a HLL, as C. This familiarity will enable them to kick start into the topic, while comparing their program with an actual one, such as LEX. That would make them aware of their shortcomings in terms of the breadth and depth of the subject of lexical analyser, for a real programming language C.

It is hoped that this will make them improve upon their version, by goading them to follow the industry standards.

Learning Outcomes:

- To understand how to design a lexical analyzer for a given language

Source Code

```
KEYWORDS = {
    "int", "float", "char", "double", "void",
    "if", "else", "while", "for", "do", "switch", "case", "default",
    "break", "continue", "return", "goto",
    "struct", "union", "enum", "typedef",
    "const", "volatile", "static", "extern", "auto", "register"
}
def tokenize(code):
    tokens = []

    # Split by whitespace and punctuation

    import re
    words = re.findall(r"[A-Za-z_][A-Za-z0-9_]*|\d+|[^\s]", code)

    for word in words:
        if word in KEYWORDS:
            tokens.append(("KEYWORD", word))
        elif re.match(r"[A-Za-z_][A-Za-z0-9_]*", word):
            tokens.append(("IDENTIFIER", word))
        elif re.match(r"\d+", word):
            tokens.append(("NUMBER", word))
        else:
            tokens.append(("SYMBOL", word))
    return tokens
# Example usage
code = "int x = 5; if (x > 0) return x;"
tokens = tokenize(code)
for token in tokens:
    print(token)
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
('KEYWORD', 'int')
('IDENTIFIER', 'x')
('SYMBOL', '=')
('NUMBER', '5')
('SYMBOL', ';')
('KEYWORD', 'if')
('SYMBOL', '(')
('IDENTIFIER', 'x')
('SYMBOL', '>')
('NUMBER', '0')
('SYMBOL', ')')
('KEYWORD', 'return')
('IDENTIFIER', 'x')
('SYMBOL', ';')
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 3:

Experiment Title: Automating Lexical Analyzer Generation Using LEX Tool.

Aim/Objective:

The aim of this lab session is to make the students conversant with the details of the LEX tool, which is an off-the-shelf tool for building scanners.

Description:

In this lab session, students will learn how to use LEX to build for a given smallish subset of C, or any other language, using the given off-the-shelf tool.

Pre-Requisites:

Students should have a basic understanding of lexical analysis process, C language.

```
#program 3:
%%writefile program.l
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return"|"break"|"continue"|"switch"|"case" {
printf("'%s' is a Keyword\n", yytext);
}
[a-zA-Z_][a-zA-Z0-9_]* {
printf("'%s' is an Identifier\n", yytext);
}

[0-9]+ {
printf("'%s' is a Number\n", yytext);
}

[ \t\n] {
/* Ignore whitespace, tabs, and newlines */
}
. {
printf("'%s' is an Unknown Character/Operator\n", yytext);
}
%%
int main() {
printf("Enter code snippet to analyze (Press Ctrl+D to stop):\n");
yylex();
return 0;
}

int yywrap() {
return 1;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Installation steps:
%%shell
sudo apt-get update && sudo apt-get install -y flex
running command:
%%shell
lex -l program.l
gcc lex.yy.c
./a.out

```
Enter code snippet to analyze (Press Ctrl+D to stop):
int
'int' is a Keyword
2
'2' is a Number
.
'.' is an Unknown Character/Operator
/n
'/' is an Unknown Character/Operator
'n' is an Identifier
akki
'akki' is an Identifier
void
'void' is a Keyword
number
'number' is an Identifier
int
'int' is a Keyword
for
'for' is a Keyword
file
'file' is an Identifier
stdio
'stdio' is an Identifier
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Program No:4
WRITE A C PROGRAM TO IMPLEMENT RECURSIVE DECENT PARSER FOR GRAMMAR:-

Aim

To write a C program to implement a **Recursive Descent Parser** and check whether the given input string is accepted by the grammar.

Description

A Recursive Descent Parser is a simple **top-down parsing method**. It uses different functions for different grammar rules. The program reads the input and checks whether it follows the given grammar. If it follows the grammar, the string is **accepted**; otherwise, it is **rejected**.

Pre-requisites

- Basic knowledge of **C programming**
- Basic knowledge of **Compiler Design**
- Understanding of **grammar and production rules**
- Knowledge of **terminals and non-terminals**
- Basic understanding of **recursion**
- Basic idea of **parsing**

```
%%writefile program1.c
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int i,err;
char inp[10];
void E();
void E1();
void T();
void T1();
void F();
void main(){
  i=0,err=0;
  printf("Enter String: ");
  scanf("%s",inp);
  E();
  if(strlen(inp)==i && err==0)
    printf("\t\t*****STRING ACCEPTED!!!!!!!!!*****\n\n");
  else
    printf("\t\t*****STRING NOT
ACCEPTED!!!!*****\n\n");
}
void E(){
  T();
  E1();
}
void E1(){
  if(inp[i]=='+'){
    i++;
    T();
    E1();
}
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
void T(){
  F();
  T1();
}
void T1(){
  if(inp[i]=='*'){
    i++;
    F();
    T1();
  }
}
void F(){
  if(isalnum(inp[i]))
    i++;
  else if(inp[i]=='('){
    i++;
    E();
    if(inp[i]==')')
    i++;
    else
    err=1;
}
else
err=1;
}
```

```
Overwriting program1.c
```

```
!gcc program1.c -o program1
!./program1
```

```
Enter String: str
                *****STRING NOT ACCEPTED!!!!*****
```

Session 4.1:

Experiment Title: Elimination of Left Recursion from given CFG (Context Free Grammar)

Aim/Objective:

The aim of this lab session is to introduce students to solution of the left recursive grammars, and the issue of infinite loop, associated with its top-down parser.

Description:
In this lab session, students will learn about CFG, and remove the problem of left recursion.

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Pre-Requisites:

Students should have a basic understanding of top-down parsing, and how it is performed.

```
#program 5:: Elimination of Left Recursion from given CFG
%%writefile kkr.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h> // For EXIT_FAILURE

int main() {
    char rule[100];

    // 1. Accept standard production rule format
    printf("Enter production rule (e.g., A->Aa|b): ");
    if (scanf("%99s", rule) != 1) return EXIT_FAILURE; // Use EXIT_FAILURE

    char lhs = rule[0]; // Left-hand side non-terminal character

    // 2. Identify direct left recursion layout (e.g., A -> A...)
    if (rule[1] == '-' && rule[2] == '>' && rule[3] == lhs) {
        char *pipe_ptr = strchr(rule, '|'); // Correct: use char *

        if (pipe_ptr != NULL) {
            printf("\nOriginal Rule: %s\n", rule);
            printf("\nRules After Eliminating Left Recursion:\n");

            // 3. Print the first new rule: A -> beta A'
            printf("%c -> ", lhs);
            // Print characters from after '|' until the end of the string
            for (char *p = pipe_ptr + 1; *p != '\0'; p++) {
                putchar(*p);
            }
            printf("%c'\n", lhs);

            // 4. Extract and print the alpha rule: A' -> alpha A' | e
            printf("%c' -> ", lhs);
            // Print characters from &rule[4] (start of alpha) up to '|'
            for (char *ptr_alpha = &rule[4]; ptr_alpha < pipe_ptr; ptr_alpha++) { // Correct: use char *
                putchar(*ptr_alpha);
            }
            printf("%c' | e\n", lhs);
        } else {
            printf("\nFormat Error: Missing option separator pipeline ('|').\n");
        }
    } else {
        printf("\nNo direct left recursion detected in this rule.\n");
    }
    return 0;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
!gcc kkr.c -o kkr
!./kkr

Enter production rule (e.g., A->Aa|b): E->E+T|T

Original Rule: E->E+T|T

Rules After Eliminating Left Recursion:
E -> TE'
E' -> +TE' | e
```

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 5:

Experiment Title: Implement Program to compute the FIRST set from the CFG.

Aim/Objective:

To make the student conversant with the predictive parser (top-down) need to know the FIRST set, in order to parse without backtracking.

Description:

The experiment involves the following steps:

i. Understand the working of a top-down parser.
ii. How is there the need to backtrack in the normal top-down search.
iii. Understand why the given parsing, needs for correct prediction the FIRST set.
iv. How to compute the FIRST set for the different grammar symbols.

Pre-Requisites:

Students should have a basic understanding of top-down search process, and the left-to-right derivation that can be derived from that in parsing, with left side symbol of a production replaced by the rhs string.

```
#program 5:: Elimination of Left Recursion from given CFG
%%writefile kkr.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h> // For EXIT_FAILURE

int main() {
    char rule[100];

    // 1. Accept standard production rule format
    printf("Enter production rule (e.g., A->Aa|b): ");
    if (scanf("%99s", rule) != 1) return EXIT_FAILURE; // Use EXIT_FAILURE

    char lhs = rule[0]; // Left-hand side non-terminal character

    // 2. Identify direct left recursion layout (e.g., A -> A...)
    if (rule[1] == '-' && rule[2] == '>' && rule[3] == lhs) {
        char *pipe_ptr = strchr(rule, '|'); // Correct: use char *

        if (pipe_ptr != NULL) {
            printf("\nOriginal Rule: %s\n", rule);
            printf("\nRules After Eliminating Left Recursion:\n");
```

```
// 3. Print the first new rule: A -> beta A'
          printf("%c -> ", lhs);
          // Print characters from after '|' until the end of the string
          for (char *p = pipe_ptr + 1; *p != '\0'; p++) {
             putchar(*p);
          }
          printf("%c'\n", lhs);

          // 4. Extract and print the alpha rule: A' -> alpha A' | e
          printf("%c' -> ", lhs);
          // Print characters from &rule[4] (start of alpha) up to '|'
          for (char *ptr_alpha = &rule[4]; ptr_alpha < pipe_ptr; ptr_alpha++) { // Correct: use char *
             putchar(*ptr_alpha);
          }
          printf("%c' | e\n", lhs);
       } else {
          printf("\nFormat Error: Missing option separator pipeline ('|').\n");
       }
    } else {
       printf("\nNo direct left recursion detected in this rule.\n");
    }
    return 0;
}
```

Output:

```
Overwriting kkr.c
```

```
!gcc kkr.c -o kkr
!./kkr
```

```
Enter production rule (e.g., A->Aa|b): E->E+T|T

Original Rule: E->E+T|T

Rules After Eliminating Left Recursion:
E -> TE'
E' -> +TE' | e
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 6 :

Experiment Title: Implement a program to compute FOLLOW Set.

Aim/Objective:

To make the student conversant with the predictive parser (top-down) need to know the FOLLOW set, in order to parse without backtracking.

Description:

The experiment involves the following steps:

v. Understand the situations where there is need to compute the FOLLOW set, for a top-down parser. Why top-down search might need to backtrack, in absence of the FOLLOW set.
vi. How to compute the FOLLOW set.

Pre-Requisites:

Students should have a basic understanding of top-down search process, and the left-to-right derivation that can be derived from that in parsing, with left side symbol of a production replaced by the RHS string.

```
%%writefile follow.c
#include <stdio.h>
#include <string.h>
char production[20][20];
int n;
int isNonTerminal(char ch) {
   return ch >= 'A' && ch <= 'Z';
}
void findFollow(char symbol) {
   int i, j;
   if (symbol == production[0][0]) {
      printf("$ ");
   }
   for (i = 0; i < n; i++) {
      for (j = 3; production[i][j] != '\0'; j++) {
         if (production[i][j] == symbol) {
            if (production[i][j + 1] != '\0') {
               char next = production[i][j + 1];
               if (!isNonTerminal(next)) {
                  if (next != 'e')
                     printf("%c ", next);
               }
            }
```

```
else {
                if (production[i][0] != symbol)
                    findFollow(production[i][0]);
            }
        }
    }
}

int main() {
    char symbol;

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (e.g., S->AB):\n");

    for (int i = 0; i < n; i++) {
        scanf("%s", production[i]);
    }

    printf("\nEnter non-terminal to find FOLLOW: ");
    scanf(" %c", &symbol);

    printf("\nFOLLOW(%c) = { ", symbol);

    findFollow(symbol);

    printf("}\n");

    return 0;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
%%shell
gcc kkr.c
./a.out
```

```
Enter number of productions: 3
Enter productions (use # for epsilon):
S->AB A->a B->b

FIRST:
FIRST(S) = {>}
FIRST(A) = {>}
FIRST(B) = {>}

FOLLOW:
FOLLOW(S) = {$}
FOLLOW(A) = {>}
FOLLOW(B) = {$}

LL(1) PARSING TABLE
-------------------

S : [S,> -> >AB]
A : [A,> -> >a]
B : [B,> -> >b]
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 7:

Experiment Title: Implement program for constructing LL(1) parsing.

```
%%writefile kkr.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 20

char prod[MAX][20];
char first[26][20], follow[26][20];
int n;

void add(char *s, char c)
{
   if (!strchr(s, c)) {
      int l = strlen(s);
      s[l] = c;
      s[l + 1] = '\0';
   }
}

int isNT(char c)
{
   return (c >= 'A' && c <= 'Z');
}

void FIRST(char nt)
{
   int i, j;
   for (i = 0; i < n; i++) {
      if (prod[i][0] != nt)
         continue;

      char *rhs = prod[i] + 2;

      if (rhs[0] == '#') {
         add(first[nt - 'A'], '#');
      }
      else if (!isNT(rhs[0])) {
         add(first[nt - 'A'], rhs[0]);
      }
      else {
         char x = rhs[0];

         if (first[x - 'A'][0] == '\0')
            FIRST(x);

         for (j = 0; first[x - 'A'][j]; j++)
            add(first[nt - 'A'], first[x - 'A'][j]);
      }
   }
}
```

```
void FOLLOW()
{
   int i, j, k;

   add(follow[prod[0][0] - 'A'], '$');

   for (k = 0; k < n; k++) {
      for (i = 0; i < n; i++) {
         char A = prod[i][0];
         char *rhs = prod[i] + 2;

         for (j = 0; rhs[j]; j++) {
            if (isNT(rhs[j])) {
               char B = rhs[j];

               if (rhs[j + 1] != '\0') {
                  char x = rhs[j + 1];

                  if (!isNT(x))
                     add(follow[B - 'A'], x);
                  else {
                     for (int m = 0; first[x - 'A'][m]; m++)
                        if (first[x - 'A'][m] != '#')
                           add(follow[B - 'A'],
                              first[x - 'A'][m]);
                  }
               }
               else {
                  for (int m = 0; follow[A - 'A'][m]; m++)
                     add(follow[B - 'A'],
                        follow[A - 'A'][m]);
               }
            }
         }
      }
   }
}
int main()
{
   int i, j;
   printf("Enter number of productions: ");
   scanf("%d", &n);
   printf("Enter productions (use # for epsilon):\n");
   for (i = 0; i < n; i++)
      scanf("%s", prod[i]);
   /* FIRST */
   for (i = 0; i < n; i++)
      FIRST(prod[i][0]);
/* FOLLOW */
   FOLLOW();
   printf("\nFIRST:\n");
   for (i = 0; i < n; i++) {
      char A = prod[i][0];
      if (i == 0 || prod[i - 1][0] != A)
         printf("FIRST(%c) = {%s}\n",
               A, first[A - 'A']);
   }
```

```
/* LL(1) Parsing Table */
    printf("\nLL(1) PARSING TABLE\n");
    printf("--------------------\n");

    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i > 0 && prod[i - 1][0] == A)
            continue;

        printf("\n%c : ", A);

        for (j = 0; j < n; j++) {
            if (prod[j][0] != A)
                continue;

            char *rhs = prod[j] + 2;

            if (rhs[0] == '#') {
                for (int k = 0; follow[A - 'A'][k]; k++)
                    printf("[%c,# -> #] ",
                           follow[A - 'A'][k]);
            }
            else if (!isNT(rhs[0])) {
                printf("[%c,%c -> %s] ",
                       A, rhs[0], rhs);
            }
            else {
                char B = rhs[0];

                for (int k = 0; first[B - 'A'][k]; k++)
                    if (first[B - 'A'][k] != '#')
                        printf("[%c,%c -> %s] ",
                               A, first[B - 'A'][k], rhs);
            }
        }
    }
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
Overwriting kkr.c
```

```
%%shell
gcc kkr.c
./a.out
```

```
Enter number of productions: 7
Enter productions (use # for epsilon):
E=TR R=+TR R=# T=FY Y=*FY Y=# F=(E) F=i

FIRST:
FIRST(E) = {(}
FIRST(R) = {+#}
FIRST(T) = {(}
FIRST(Y) = {*#}
FIRST(F) = {(}

FOLLOW:
FOLLOW(E) = {$)}
FOLLOW(R) = {$)}
FOLLOW(T) = {+}
FOLLOW(Y) = {+}
FOLLOW(F) = {*}

LL(1) PARSING TABLE
-------------------

E : [E,( -> TR]
R : [R,+ -> +TR] [$,# -> #] [),# -> #]
T : [T,( -> FY]
Y : [Y,* -> *FY] [+,# -> #]
F : [F,( -> (E)]
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured_____out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 8:

Experiment Title: Implement a program for shift-reduce parser.

```
%%writefile shift_reduce.c
#include <stdio.h>
#include <string.h>
int main()
{
   char stack[50], input[50];
   int top = -1, i = 0;
   printf("Enter input string: ");
   scanf("%s", input);
   printf("\nSTACK\tINPUT\tACTION\n");
   while (input[i] != '\0') {
      stack[++top] = input[i++];
      stack[top + 1] = '\0';
      printf("%s\t%s\tSHIFT\n", stack, input + i);
      if (top >= 1 &&
         stack[top - 1] == 'i' &&
         stack[top] == 'd') {
         top -= 1;
         stack[top] = 'E';
         stack[top + 1] = '\0';
         printf("%s\t%s\tREDUCE E->id\n",
               stack, input + i);
      }
      if (top >= 2 &&
         stack[top - 2] == 'E' &&
         stack[top - 1] == '+' &&
         stack[top] == 'E') {
         top -= 2;
         stack[top] = 'E';
         stack[top + 1] = '\0';
         printf("%s\t%s\tREDUCE E->E+E\n",
               stack, input + i);
      }
   }
   if (strcmp(stack, "E") == 0)
      printf("\nString Accepted!\n");
   else
      printf("\nString Rejected!\n");
   return 0;
}
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
Overwriting shift_reduce.c
```

```
%%shell
gcc shift_reduce.c
./a.out
```

```
Enter input string: id+id

STACK   INPUT   ACTION
i       d+id    SHIFT
id      +id     SHIFT
E       +id     REDUCE E->id
E+      id      SHIFT
E+i     d       SHIFT
E+id            SHIFT
E+E             REDUCE E->id
E               REDUCE E->E+E

String Accepted!
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 9:

Experiment Title: Implement Program for LR parser.

```
%%writefile lr_parser.c
#include <stdio.h>
#include <string.h>
char stack[100];
int top = -1;
void push(char c)
{
   stack[++top] = c;
   stack[top + 1] = '\0';
}
void pop(int n)
{
   top -= n;
   stack[top + 1] = '\0';
}
int main()
{
   char input[100];
   int i = 0;
   printf("Enter input string: ");
   scanf("%s", input);
   printf("\nSTACK\tINPUT\tACTION\n");
   while (input[i] != '\0') {
      /* Shift */
      if (input[i] == 'i' && input[i + 1] == 'd') {
         push('i');
         push('d');
         i += 2;
         printf("%s\t%s\tSHIFT id\n",
               stack, input + i);
         /* Reduce F -> id */
         pop(2);
         push('F');
         printf("%s\t%s\tREDUCE F->id\n",
               stack, input + i);
         /* Reduce T -> F */
         pop(1);
         push('T');
         printf("%s\t%s\tREDUCE T->F\n",
               stack, input + i);
         /* Reduce E -> T */
         if (input[i] == '\0' || input[i] == '+') {
            pop(1);
            push('E');
            printf("%s\t%s\tREDUCE E->T\n",
                  stack, input + i);
         }
      }
```

```
else if (input[i] == '+') {
        push('+');
        i++;
        printf("%s\t%s\tSHIFT +\n",
            stack, input + i);
    }
    else {
        printf("\nString Rejected!\n");
        return 0;
    }
  }
  if (strcmp(stack, "E") == 0)
    printf("\nString Accepted!\n");
  else
    printf("\nString Rejected!\n");
  return 0;
}
```

```
Overwriting lr_parser.c
```

```
%%shell
gcc lr_parser.c
./a.out
```

```
Enter input string: id+id*id

STACK   INPUT   ACTION
id      +id*id  SHIFT id
F       +id*id  REDUCE F->id
T       +id*id  REDUCE T->F
E       +id*id  REDUCE E->T
E+      id*id   SHIFT +
E+id    *id     SHIFT id
E+F     *id     REDUCE F->id
E+T     *id     REDUCE T->F

String Rejected!
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 10:

Experiment Title: Implement a System to generate and interpret quadruples.

```
%%writefile program10.c
#include <stdio.h>
#include <string.h>

#define MAX 20

struct Quad {
    char op[5];
    char arg1[20];
    char arg2[20];
    char result[20];
};

struct Quad q[MAX];
int n = 0;

int value(char name[]) {
    if (strcmp(name, "a") == 0) return 10;
    if (strcmp(name, "b") == 0) return 20;
    if (strcmp(name, "c") == 0) return 5;
    if (strcmp(name, "d") == 0) return 4;

    return 0;
}

void generate(char op[], char a1[],
              char a2[], char res[]) {
    strcpy(q[n].op, op);
    strcpy(q[n].arg1, a1);
    strcpy(q[n].arg2, a2);
    strcpy(q[n].result, res);
    n++;
}
int main() {
  int i, x, y, z;
  int temp[ MAX ] = {0};

  // Generate quadruples for:
  // a = b + c * d

  generate("*", "c", "d", "t1");
  generate("+", "b", "t1", "t2");
  generate("=", "t2", "-", "a");

  printf("\nQUADRUPLE TABLE\n");
  printf("-----------------------------------\n");
  printf("OP\tARG1\tARG2\tRESULT\n");
  printf("-----------------------------------\n");

  for(i = 0; i < n; i++) {
      printf("%s\t%s\t%s\t%s\n",
      q[i].op, q[i].arg1,
      q[i].arg2, q[i].result);
  }

  // Interpret quadruples
  printf("\nINTERPRETATION\n");

  for(i = 0; i < n; i++) {

      if(q[i].arg1[0] == 't')
          x = temp[q[i].arg1[1] - '1'];
      else
          x = value(q[i].arg1);

      if(q[i].arg2[0] == 't')
          y = temp[q[i].arg2[1] - '1'];
      else
          y = value(q[i].arg2);

      if(strcmp(q[i].op, "*") == 0)
          z = x * y;
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
        else if(strcmp(q[i].op, "+") == 0)
            z = x + y;

        else if(strcmp(q[i].op, "=") == 0)
            z = x;

        else
            z = 0;

        if(q[i].result[0] == 't')
            temp[q[i].result[1] - '1'] = z;

        printf("%s = %d\n", q[i].result, z);
    }

    return 0;
}
```

```
Overwriting program10.c
```

```
%%shell
gcc program10.c -o program10
./program10
```

```
QUADRUPLE TABLE
-----------------------------------
OP      ARG1    ARG2    RESULT
-----------------------------------
*       c       d       t1
+       b       t1      t2
=       t2      -       a

INTERPRETATION
t1 = 20
t2 = 40
a = 40
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 11:

Experiment Title: Implement a simple code optimization technique.

```
%%writefile program11.c
#include <stdio.h>

int main()
{
    int a, b, c;

    // Original expressions
    printf("Before Optimization:\n");

    printf("a = 10 + 20\n");
    printf("b = 5 * 4\n");
    printf("c = a + b\n");

    // Constant Folding
    a = 10 + 20;
    b = 5 * 4;
    c = a + b;

    printf("\nAfter Optimization:\n");

    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("c = a + b\n");

    printf("\nFinal Result:\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("c = %d\n", c);

    return 0;
}
```

```
Overwriting program11.c
```

```
%%shell
gcc program11.c -o program11
./program11
```

```
Before Optimization:
a = 10 + 20
b = 5 * 4
c = a + b

After Optimization:
a = 30
b = 20
c = a + b

Final Result:
a = 30
b = 20
c = 50
```

(For Evaluator's use o

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

Session 12:

Experiment Title: Implement a module for generating intermediate code.

```
%%writefile program12.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char expr[100];
char temp[10];
int t = 1;

void generate(char op, char a, char b)
{
    printf("t%d = %c %c %c\n", t, a, op, b);
    t++;
}

int main()
{
    int i;
    char a, b, op;

    printf("Enter an expression (example: a+b*c): ");
    scanf("%s", expr);

    printf("\nIntermediate Code:\n");

    // Generate multiplication and division first
    for(i = 0; expr[i] != '\0'; i++)
    {
        if(expr[i] == '*' || expr[i] == '/')
        {
            a = expr[i-1];
            b = expr[i+1];
            op = expr[i];

            generate(op, a, b);

            // Replace processed expression with temporary
            expr[i-1] = '0' + t - 2;
            memmove(&expr[i], &expr[i+2],
                    strlen(&expr[i+2]) + 1);
```

Experiment	<TOBEFILLEDBYSTUDENT>	Student ID	<TOBEFILLEDBYSTUDENT>
Date	<TOBEFILLEDBYSTUDENT>	Student Name	<TOBEFILLEDBYSTUDENT>

```
            // Replace processed expression with temporary
            expr[i-1] = '0' + t - 2;
            memmove(&expr[i], &expr[i+2],
                    strlen(&expr[i+2]) + 1);
            i = 0;
        }
    }

    // Generate addition and subtraction
    for(i = 0; expr[i] != '\0'; i++)
    {
        if(expr[i] == '+' || expr[i] == '-')
        {
            a = expr[i-1];
            b = expr[i+1];
            op = expr[i];

            generate(op, a, b);
            i = 0;
        }
    }

    return 0;
}
```

```
Overwriting program12.c
```

```
%%shell
gcc program12.c -o program12
./program12
```

```
Enter an expression (example: a+b*c):
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator

Experiment	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Session 13:

Experiment Title: Implement basic error detection and reporting system.

```
%%writefile error_detection.c
#include <stdio.h>
#include <string.h>

int main()
{
    char code[200];
    int i, open = 0, error = 0;

    printf("===== BASIC ERROR DETECTION SYSTEM =====\n");

    printf("\nEnter a statement: ");
    fgets(code, sizeof(code), stdin);

    // Check empty input
    if (code[0] == '\n' || code[0] == '\0')
    {
        printf("\nError: Empty statement!\n");
        return 0;
    }

    // Check parentheses
    for (i = 0; code[i] != '\0'; i++)
    {
        if (code[i] == '(')
            open++;

        else if (code[i] == ')')
        {
            open--;

            if (open < 0)
            {
                printf("\nError: Unmatched closing parenthesis!\n");
                error = 1;
                break;
            }
        }
    }
```

Experiment	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

```
    if (open > 0)
    {
        printf("\nError: Missing closing parenthesis!\n");
        error = 1;
    }

    // Check semicolon
    int len = strlen(code);

    while (len > 0 &&
           (code[len-1] == '\n' || code[len-1] == ' '))
    {
        len--;
    }

    if (len > 0 && code[len-1] != ';' &&
        code[len-1] != '}')
    {
        printf("\nError: Missing semicolon!\n");
        error = 1;
    }

    // Check assignment operator
    if (strchr(code, '=') == NULL &&
        strstr(code, "printf") == NULL &&
        strstr(code, "scanf") == NULL)
    {
        printf("\nError: Assignment operator missing!\n");
        error = 1;
    }

    if (error == 0)
    {
        printf("\nNo errors detected!\n");
        printf("Statement is syntactically correct.\n");
    }

    return 0;
}
```

```
%%shell
gcc error_detection.c -o error_detection
./error_detection

===== BASIC ERROR DETECTION SYSTEM =====

Enter a statement: a = 10;

No errors detected!
Statement is syntactically correct.
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation
	Marks Secured _____ out of **50** Signature of the Evaluator